

TOPA 3급 온라인

03 데이터 처리



목차

—

pandas를 사용해 직접
데이터를 불러오고, 수정하고,
데이터에 대한 통계를 계산
해봅시다.

01 데이터 이론

02 데이터 로드

03 데이터 변환

04 데이터 요약하기



목차

—

pandas를 사용해 직접
데이터를 불러오고, 수정하고,
데이터에 대한 통계를 계산
해봅시다.

05 이상치와 결측치

06 데이터 병합

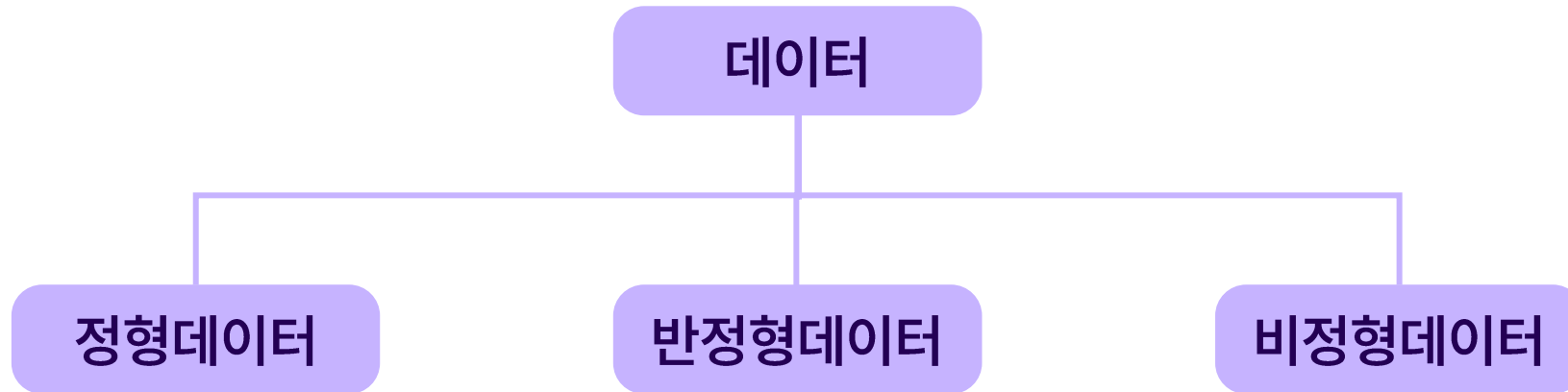
07 기초 통계

01

데이터 이론

✓ 데이터란?

- 정보를 전달하는 연속 혹은 불연속 값의 모음
- 데이터는 그 형식에 따라 크게 정형 데이터, 반정형데이터, 비정형데이터로 구분할 수 있음



④ 데이터란?

정형데이터

표준화되고 명확하게 정의되며 접근성이 높고 전통적인 데이터베이스를 이용한 처리가 용이한 데이터

예) 직원 데이터 베이스, 판매 기록 등

	A	B	C	D	E
1	Employee Database 2019				
2	Employee ID	First Name	Last Name	Designation	Salary
3	D01	William	Smith	CEO	€ 2,50,000
4	D02	Rezza	Fredd	Sr Manager	€ 1,00,000
5	D03	Mohan	Kumar	Web Develpoer	€ 50,000
6	D04	Cindy	Mac	Senior HR	€ 50,000
7	D05	Vince	Jose	HR Consultant	€ 30,000

✓ 데이터란?

반정형데이터

일정 수준의 표준화와 구조가 정의되어 있으나 구조에 유연성을 가지는 데이터

예) JSON, XML 등



```
<!DOCTYPE html>
<html lang="ko" dir="ltr" itemscope itemtype="http://schema.org/SearchResultsPage" style>
  <head>
  <body id="yDmH0d" jscontroller="pjICDe" jsaction="rcuQ6b:npT2md; click:FABpgf; auxclick:FABpgf;UjQMac:.CLIENT;c0v8t:.CLIENT;keydown:.CLIENT;keyup:.CLIENT;keypress:.CLIENT;nHjqDd:.CLIENT;LhiQec:.CLIENT;qako4e:.CLIENT;GvneHb:.CLIENT;OEHo9c:.CLIENT;Vu36Xe:.CLIENT;qGw9D:.CLIENT;OHn2db:.CLIENT;wINJic:.CLIENT" class="tQj5Y ghyPEc IqBfM ecJEib EWZcud eejsDc uirfo notranslate EI1Dfe cjGgb d8Etdd LcUz9d uOat3d" data-has-header="true" data-has-footer="true" data-init="1" style="min-height: 937px;">
    <script aria-hidden="true" nonce>
```


✓ 데이터란?

비정형데이터

미리 정의된 형식이 없는 데이터를 의미하며 전통적인 데이터베이스를 이용한 정리나 처리가 어려움
(예시) 사진, 소셜 미디어 게시물 등

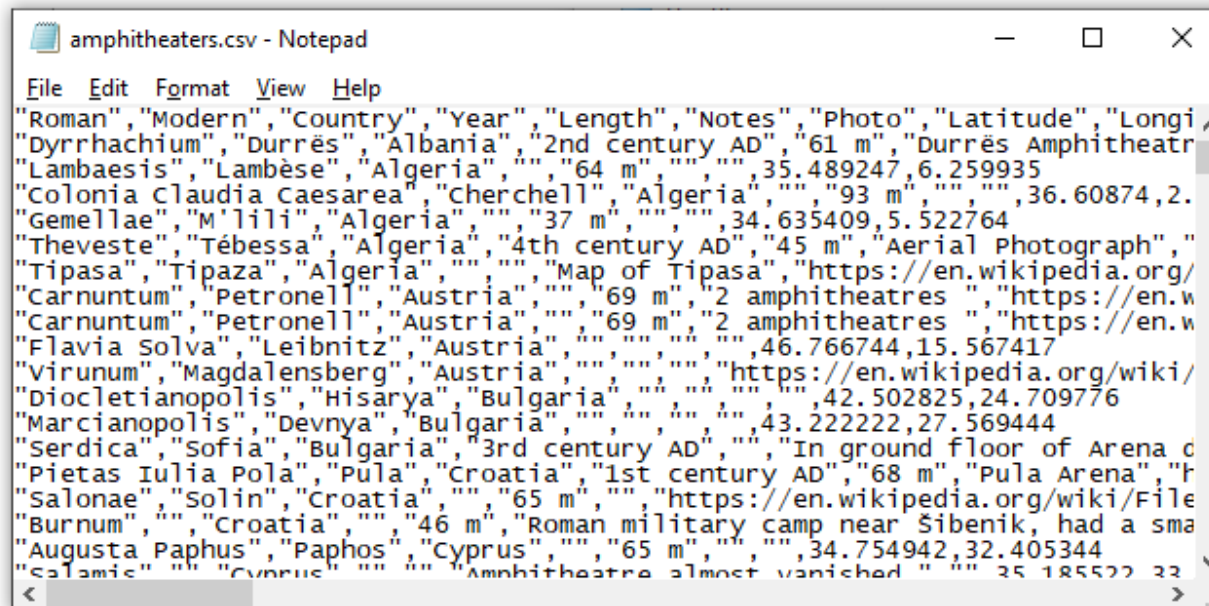


02

데이터 로드

✓ CSV (Comma-Separated Values)

- 쉼표(,)로 구분한 텍스트 데이터 및 텍스트 파일
- 저장용량이 작고 단순하며 다양한 소프트웨어 응용 프로그램 및 프로그래밍 언어에서 광범위하게 지원되기 때문에 데이터 교환에 널리 사용됨



```
amphitheatres.csv - Notepad
File Edit Format View Help
"Roman","Modern","Country","Year","Length","Notes","Photo","Latitude","Longitude"
"Dyrrhachium","Durrës","Albania","2nd century AD","61 m","Durrës Amphitheatre","https://en.wikipedia.org/wiki/File:Dyrrhachium_Amphitheatre.jpg",35.489247,19.259935
"Lambaesis","Lambèse","Algeria","3rd century AD","64 m","Lambaesis Amphitheatre","https://en.wikipedia.org/wiki/File:Lambaesis_Amphitheatre.jpg",35.489247,6.259935
"Colonia Claudia Caesarea","Cherchell","Algeria","3rd century AD","93 m","Cherchell Amphitheatre","https://en.wikipedia.org/wiki/File:Cherchell_Amphitheatre.jpg",36.60874,2.259935
"Gemellae","M'lili","Algeria","3rd century AD","37 m","Gemellae Amphitheatre","https://en.wikipedia.org/wiki/File:Gemellae_Amphitheatre.jpg",34.635409,5.522764
"Theveste","Tébessa","Algeria","4th century AD","45 m","Theveste Amphitheatre","https://en.wikipedia.org/wiki/File:Theveste_Amphitheatre.jpg",34.635409,5.522764
"Tipasa","Tipaza","Algeria","3rd century AD","69 m","Tipasa Amphitheatre","https://en.wikipedia.org/wiki/File:Tipasa_Amphitheatre.jpg",36.60874,2.259935
"Carnuntum","Petronell","Austria","1st century AD","69 m","2 amphitheatres","https://en.wikipedia.org/wiki/File:Carnuntum_Amphitheatre.jpg",46.766744,15.567417
"Flavia Solva","Leibnitz","Austria","1st century AD","69 m","2 amphitheatres","https://en.wikipedia.org/wiki/File:Flavia_Solva_Amphitheatre.jpg",46.766744,15.567417
"Virunum","Magdalensberg","Austria","1st century AD","69 m","2 amphitheatres","https://en.wikipedia.org/wiki/File:Virunum_Amphitheatre.jpg",46.766744,15.567417
"Diocletianopolis","Hisarya","Bulgaria","3rd century AD","69 m","2 amphitheatres","https://en.wikipedia.org/wiki/File:Diocletianopolis_Amphitheatre.jpg",42.502825,24.709776
"Marcianopolis","Devnya","Bulgaria","3rd century AD","69 m","2 amphitheatres","https://en.wikipedia.org/wiki/File:Marcianopolis_Amphitheatre.jpg",43.222222,27.569444
"Serdica","Sofia","Bulgaria","3rd century AD","69 m","2 amphitheatres","https://en.wikipedia.org/wiki/File:Serdica_Amphitheatre.jpg",42.502825,24.709776
"Pietas Iulia Pola","Pula","Croatia","1st century AD","68 m","Pula Arena","https://en.wikipedia.org/wiki/File:Pietas_Iulia_Pola_Amphitheatre.jpg",45.502825,13.709776
"Salonae","Solin","Croatia","1st century AD","65 m","Salonae Amphitheatre","https://en.wikipedia.org/wiki/File:Salonae_Amphitheatre.jpg",45.502825,13.709776
"Burnum","","Croatia","1st century AD","46 m","Roman military camp near Šibenik, had a small amphitheatre","https://en.wikipedia.org/wiki/File:Burnum_Amphitheatre.jpg",45.502825,13.709776
"Augusta Paphus","Paphos","Cyprus","1st century AD","65 m","Augusta Paphus Amphitheatre","https://en.wikipedia.org/wiki/File:Augusta_Paphus_Amphitheatre.jpg",34.754942,32.405344
"Salamis","","Cyprus","1st century AD","65 m","Augusta Paphus Amphitheatre","https://en.wikipedia.org/wiki/File:Salamis_Amphitheatre.jpg",34.754942,32.405344
```

✓ CSV 모듈

- CSV 모듈은 파이썬의 내장 라이브러리로 CSV 파일을 다루는 기능을 제공함
- 'csv.reader()'를 통해 csv 파일을 입력으로 받아들이고 파일의 각 행을 순회하면 데이터를 읽음

```
import csv

# CSV 파일 열기
with open('data.csv', 'r') as file:
    # CSV reader 객체 생성
    csv_reader = csv.reader(file)

    # CSV 파일의 각 행 순회
    for row in csv_reader:
        print(row) # 각 행의 값들은 리스트로 표현됩니다.
```

Result

```
[Name, Age, Occupation]
[John, 30, Engineer]
[Sarah, 25, Doctor]
[Michael, 28, Teacher]
```

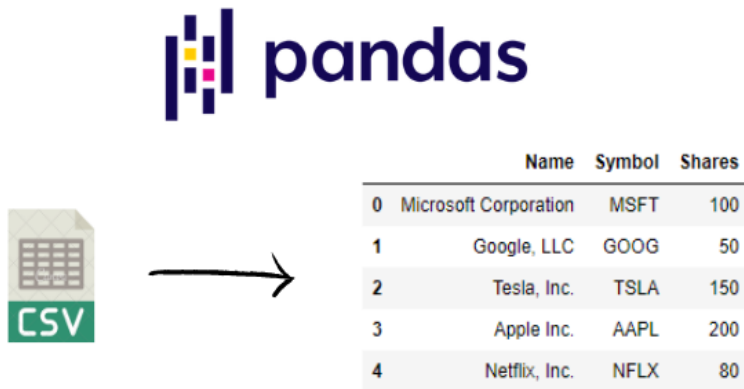
✓ Pandas 라이브러리

- Pandas는 데이터 조작 및 분석을 위해 설계된 강력한 파이썬 라이브러리
- 기존 CSV 모듈에서 수동으로 처리해야 했던 파싱 및 데이터 변환을 자동으로 처리하여 구조화된 표 형태의 데이터를 작업하는데 유용함



☑ Pandas 라이브러리의 장점

- 또한, 'pandas.read_csv()' 함수는 CSV파일을 DataFrame로 반환하는데, 이는 강력한 데이터 조작 기능을 제공함



```
import pandas as pd
```

```
# CSV 파일을 DataFrame으로 로드  
df = pd.read_csv('data.csv')
```

```
# DataFrame 출력  
print(df)
```

Result

Name	Age	Occupation
John	30	Engineer
Sarah	25	Doctor
Michael	28	Teacher

03

데이터 변환

☑ 기존 데이터 기반 새로운 열 추가

```
#'Adult' 열 추가
df['Adult'] = df['Age'] >= 18
print("Adult열 추가!")
print(df)
print()

#'Birth' 열 추가
df['Birth(years)'] = 2023 - df['Age']
print('Birth열 추가!')
print(df)
```

df			
	Name	Age	Occupation
0	John	31	Engineer
1	Sarah	26	Doctor
2	Michael	29	Teacher

Result					
Adult열 추가!					
	Name	Age	Occupation	Adult	
0	John	31	Engineer	True	
1	Sarah	26	Doctor	True	
2	Michael	29	Teacher	True	
Birth열 추가!					
	Name	Age	Occupation	Adult	Birth(years)
0	John	31	Engineer	True	1992
1	Sarah	26	Doctor	True	1997
2	Michael	29	Teacher	True	1994

✓ 범주형 데이터 → 수치형 데이터 변환

df					
	Name	Age	Occupation	Adult	Birth(years)
0	John	31	Engineer	True	1993
1	Sarah	26	Doctor	True	1998
2	Michael	29	Teacher	True	1995

```
import numpy as np
df['Adult'] = np.where(df['Adult']==True, 1, 0)
print(df)
```

Result					
	Name	Age	Occupation	Adult	Birth(years)
0	John	31	Engineer	1	1992
1	Sarah	26	Doctor	1	1997
2	Michael	29	Teacher	1	1994

✓ rename()

df					
	Name	Age	Occupation	Adult	Birth(years)
0	John	31	Engineer	1	1993
1	Sarah	26	Doctor	1	1998
2	Michael	29	Teacher	1	1995

```
import pandas as pd
```

```
df = df.rename(columns={'Age': 'Age(나이)'})
```

```
print(df)
```

Result					
	Name	Age(나이)	Occupation	Adult	Birth(years)
0	John	31	Engineer	1	1992
1	Sarah	26	Doctor	1	1997
2	Michael	29	Teacher	1	1994

✓ rename()

df					
	Name	Age	Occupation	Adult	Birth(years)
0	John	31	Engineer	1	1993
1	Sarah	26	Doctor	1	1998
2	Michael	29	Teacher	1	1995

```
df = df.rename(index={1: 'one'})
print(df)
```

Result					
	Name	Age(나이)	Occupation	Adult	Birth(years)
0	John	31	Engineer	1	1992
one	Sarah	26	Doctor	1	1997
2	Michael	29	Teacher	1	1994

☑ 조건에 따른 행 필터링

df					
	Name	Age	Occupation	Adult	Birth(years)
0	John	31	Engineer	1	1993
1	Sarah	26	Doctor	1	1998
2	Michael	29	Teacher	1	1995

```
import pandas as pd
```

```
#열 조건에 따른 필터링
```

```
filtered_df = df[df['Age(나이)'] >= 27]
```

```
print(filtered_df)
```

```
print('-' * 55)
```

```
#두 가지 조건을 이용한 필터링
```

```
filtered_two_df = df[(df['Age(나이)'] >= 27) & (df['Occupation']=='Engineer')]
```

```
print(filtered_two_df)
```

Result					
	Name	Age(나이)	Occupation	Adult	Birth(years)
0	John	31	Engineer	1	1992
2	Michael	29	Teacher	1	1994

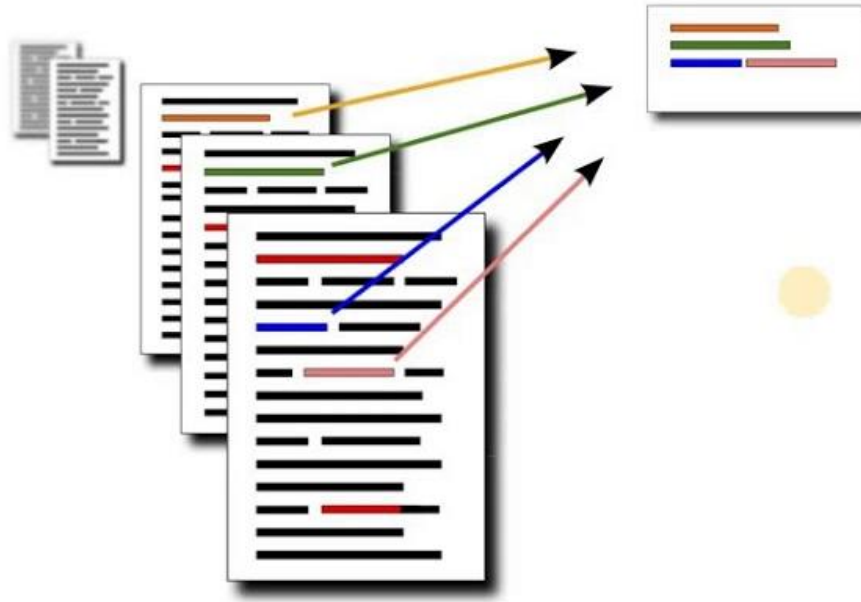
	Name	Age(나이)	Occupation	Adult	Birth(years)
0	John	31	Engineer	1	1992

04

데이터 요약하기

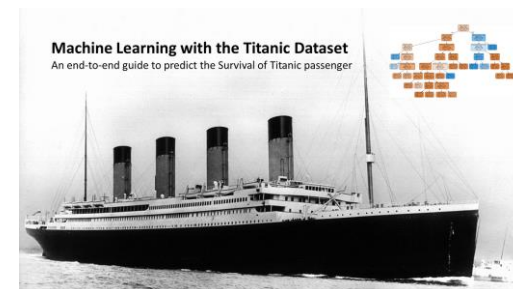
☑ 데이터 요약하기

- 요약된 데이터 정보를 통해 데이터의 분포 및 주요 경향을 빠르게 이해할 수 있으며, 이를 통해 데이터간 비교, 정리, 검증 등이 가능해짐



✔ titanic 데이터세트

- 타이타닉 데이터셋은 데이터 분석과 머신러닝 세계에서 가장 잘 알려지고 널리 사용되는 데이터셋 중 하나
- 1912년에 항해 중에 침몰한 RMS 타이타닉호의 승객에 관한 정보를 포함하고 있음
- 일반적으로 다양한 특징을 기반으로 승객이 침몰에서 살아남았을지 예측하는 모델을 만드는 작업에 사용
- 타이타닉 데이터셋 다운로드: <https://www.kaggle.com/c/titanic/data>



✔ titanic 데이터셋 불러오기

```
import pandas as pd

df = pd.read_csv('./train.csv')
print(df)
```

```
   PassengerId  Survived  Pclass \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3
..          ...         ...     ...
886          887         0       2
887          888         1       1
888          889         0       3
889          890         1       1
890          891         0       3

   Name                               Sex  Age  SibSp \
0  Braund, Mr. Owen Harris             male  22.0    1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0    1
2  Heikkinen, Miss. Laina              female  26.0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0    1
4  Allen, Mr. William Henry            male  35.0    0
..          ...         ...     ...     ...
886  Montvila, Rev. Juozas              male  27.0    0
887  Graham, Miss. Margaret Edith        female  19.0    0
888  Johnston, Miss. Catherine Helen "Carrie"    female   NaN    1
889   Behr, Mr. Karl Howell              male  26.0    0
890  Dooley, Mr. Patrick                 male  32.0    0
```

```
   Parch  Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0    PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0   113803  53.1000  C123      S
4      0   373450   8.0500   NaN      S
..     ...     ...     ...     ...
886     0   211536  13.0000   NaN      S
887     0   112053  30.0000  B42      S
888     2    W./C. 6607  23.4500   NaN      S
889     0   111369  30.0000  C148      C
890     0   370376   7.7500   NaN      Q

[891 rows x 12 columns]
```

✓ head()

- 전체 데이터의 첫 5줄에 대한 정보를 제공하며, 5줄 미만인 경우 모든 정보 제공

```
import pandas as pd

df.head()
```

Index	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.925	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05	NaN	S

✓ describe()

- 데이터 프레임의 통계 요약을 가져오는 데 사용됨
- DataFrame의 경우 열에 대한 요약이 수행되며, 결측치(NaN)은 제외되고 데이터 요약이 수행됨

```
import pandas as pd

df.describe()
```

index	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.0	891.0	891.0	714.0	891.0	891.0	891.0
mean	446.0	0.3838383838383838	2.308641975308642	29.69911764705882	0.523007856341896	0.38159371492704824	32.204207968574636
std	257.3538420152301	0.4865924542648575	0.836071240977049	14.526497332334042	1.1027434322934317	0.8060572211299483	49.6934285971809
min	1.0	0.0	1.0	0.42	0.0	0.0	0.0
25%	223.5	0.0	2.0	20.125	0.0	0.0	7.9104
50%	446.0	0.0	3.0	28.0	0.0	0.0	14.4542
75%	668.5	1.0	3.0	38.0	1.0	0.0	31.0
max	891.0	1.0	3.0	80.0	8.0	6.0	512.3292

✓ info()

- 데이터 수, 열 이름, 데이터 유형, 인덱스 범위 및 메모리 사용량과 같이 데이터 프레임에 대해 알아야 할 모든 정보를 보여줌

```
import pandas as pd

df.info(verbose=True)
```

Result

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

✓ shape

- DataFrame의 차원을 나타내는 튜플을 반환
- 반환 값의 첫 번째 요소는 행의 수이고, 두 번째 요소는 열의 수입니다.

```
print(df.shape)
```

Result

(891, 12)

④ value_counts()

```
print(df['Survived'].value_counts())
```

Result

```
0    549
```

```
1    342
```

```
Name: Survived, dtype: int64
```


✔ value_counts()

```
print(df['Pclass'].value_counts())
```

Result

```
3    491
```

```
1    216
```

```
2    184
```

```
Name: Pclass, dtype: int64
```

05

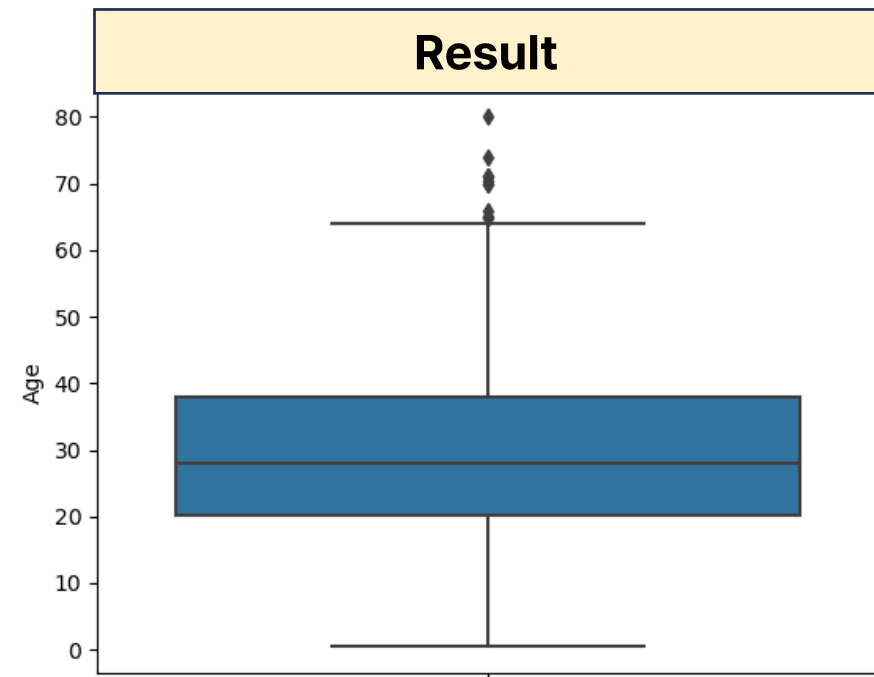
이상치와 결측치

☑ 데이터 시각화

- 데이터 시각화를 통해 이상치 찾기

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(y=df['Age'])
plt.show()
```



☑ 이상치 처리

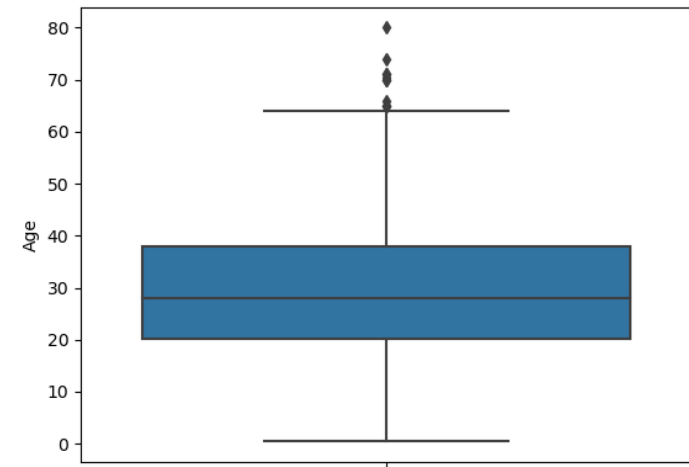
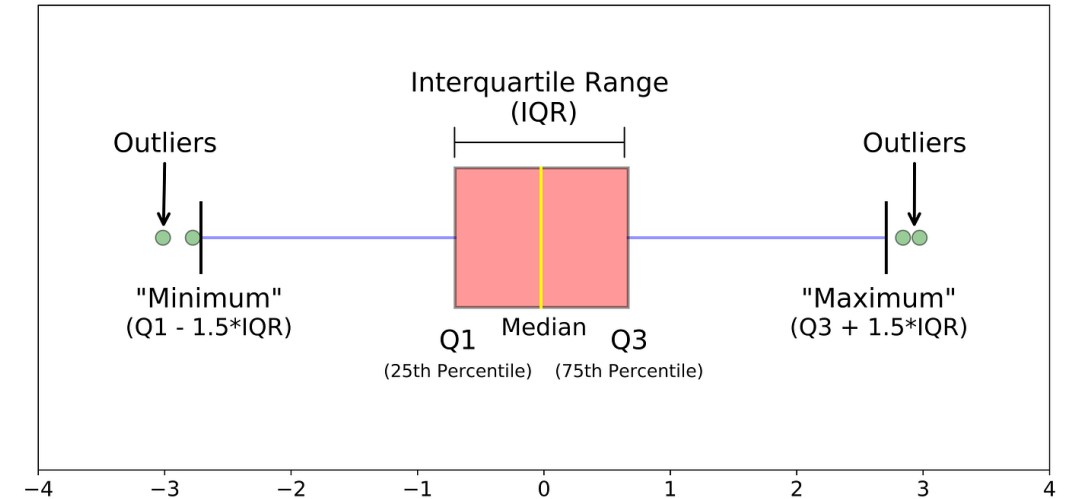
```
Q1 = df['Age'].quantile(q=0.25)
Q3 = df['Age'].quantile(q=0.75)

IQR = Q3 - Q1
M = Q3 + 1.5*IQR

print(M)
```

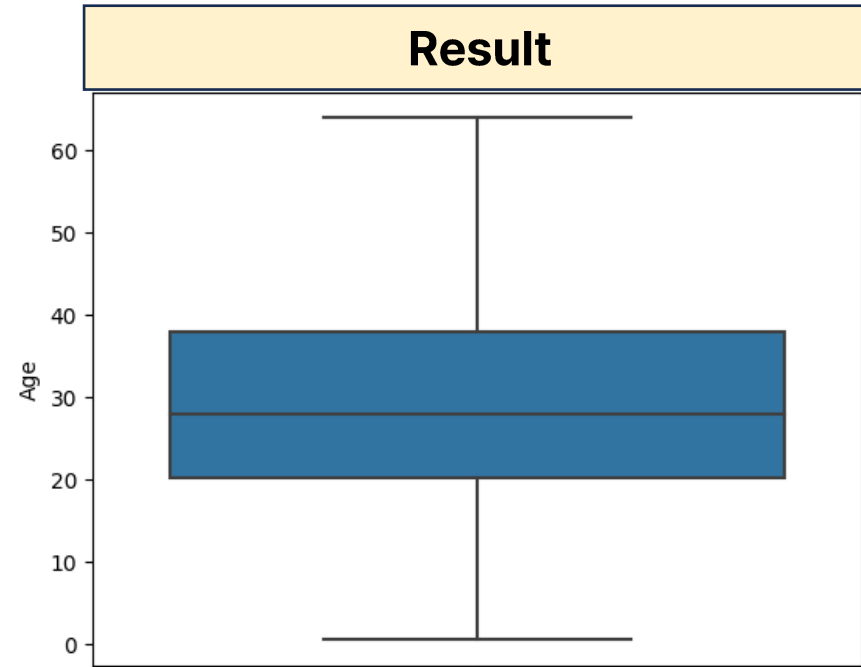
Result

64.8125



☑ 이상치 처리

```
df[df['Age']>64] = 64  
  
sns.boxplot(y=df['Age'])  
plt.show()
```



④ notnull()

```
print(df.notnull())
```

Result

	PassengerId	Survived	Pclass	Name	...	Fare	Cabin	Embarked
0	True	True	True	True	...	True	False	True
1	True	True	True	True	...	True	True	True
2	True	True	True	True	...	True	False	True
3	True	True	True	True	...	True	True	True
4	True	True	True	True	...	True	False	True
..	...	...	...	...	...	...	...	...
886	True	True	True	True	...	True	False	True
887	True	True	True	True	...	True	True	True
888	True	True	True	True	...	True	False	True
889	True	True	True	True	...	True	True	True
890	True	True	True	True	...	True	False	True

```
[891 rows x 12 columns]
```

☑ notnull()

```
print(df.notnull().sum())
```

Result

PassengerId	891
Survived	891
Pclass	891
Name	891
Sex	891
Age	714
SibSp	891
Parch	891
Ticket	891
Fare	891
Cabin	210
Embarked	889
dtype:	int64

✓ fillna()

- 특정한 값으로 결측치들을 대체할 수 있음

```
print('Age mean: ', df['Age'].mean())
print('Age max: ', df['Age'].max())
print('-' * 20)
df['Age'] = df['Age'].fillna(df['Age'].mean())
print(df.notnull().sum())
```

Result

Age mean: 29.61018207282913

Age max: 64.0

```
-----
PassengerId    891
Survived        891
Pclass         891
Name           891
Sex            891
Age            891
SibSp          891
Parch          891
Ticket         891
Fare           891
Cabin         210
Embarked       889
dtype: int64
```


✓ drop()

before	
PassengerId	891
Survived	891
Pclass	891
Name	891
Sex	891
Age	891
SibSp	891
Parch	891
Ticket	891
Fare	891
Cabin	210
Embarked	889
dtype: int64	

```
df = df.drop(['Cabin'], axis=1)
print(df.notnull().sum())
```

Result	
PassengerId	891
Survived	891
Pclass	891
Name	891
Sex	891
Age	891
SibSp	891
Parch	891
Ticket	891
Fare	891
Embarked	889
dtype: int64	

✓ dropna()

- 데이터셋에서 결측치가 있는 행을 제거할 수 있음

before	
PassengerId	891
Survived	891
Pclass	891
Name	891
Sex	891
Age	891
SibSp	891
Parch	891
Ticket	891
Fare	891
Embarked	889
dtype: int64	

```
df = df.dropna()
print(df.notnull().sum())
```

Result	
PassengerId	889
Survived	889
Pclass	889
Name	889
Sex	889
Age	889
SibSp	889
Parch	889
Ticket	889
Fare	889
Embarked	889
dtype: int64	

06

데이터 병합

✓ concat()

jul_df				
	날짜	출금액	입금액	잔액
0	2023-07-01	80,000	0	100,000
1	2023-07-01	0	150,000	250,000
2	2023-07-05	75,500	0	174,500
3	2023-07-11	3,000	0	171,500
4	2023-07-11	0	15,000	186,500

aug_df				
	날짜	출금액	입금액	잔액
0	2023-08-02	0	80,000	190,000
1	2023-08-02	0	150,000	340,000
2	2023-08-05	15,000	0	325,000
3	2023-08-11	7,000	0	318,000
4	2023-08-13	0	22,000	340,000

```
import pandas as pd

jul_df = pd.read_csv('./7월.csv')
aug_df = pd.read_csv('./8월.csv')

concat_df = [jul_df, aug_df]

jul_aug_df = pd.concat(concat_df).reset_index(drop=True)
#jul_aug_df = pd.concat([jul_df, aug_df]).reset_index(drop=True)
print(jul_aug_df)
```

Result				
	날짜	출금액	입금액	잔액
0	2023-07-01	80,000	0	100,000
1	2023-07-01	0	150,000	250,000
2	2023-07-05	75,500	0	174,500
3	2023-07-11	3,000	0	171,500
4	2023-07-11	0	15,000	186,500
5	2023-08-02	0	80,000	190,000
6	2023-08-02	0	150,000	340,000
7	2023-08-05	15,000	0	325,000
8	2023-08-11	7,000	0	318,000
9	2023-08-13	0	22,000	340,000

✓ concat()

jul_df

	날짜	출금액	입금액	잔액
0	2023-07-01	80,000	0	100,000
1	2023-07-01	0	150,000	250,000
2	2023-07-05	75,500	0	174,500
3	2023-07-11	3,000	0	171,500
4	2023-07-11	0	15,000	186,500

aug_df

	날짜	출금액	입금액	잔액
0	2023-08-02	0	80,000	190,000
1	2023-08-02	0	150,000	340,000
2	2023-08-05	15,000	0	325,000
3	2023-08-11	7,000	0	318,000
4	2023-08-13	0	22,000	340,000

```
import pandas as pd
```

```
jul_df = pd.read_csv('./7월.csv')
```

```
aug_df = pd.read_csv('./8월.csv')
```

```
concat_df = [jul_df, aug_df]
```

```
jul_aug_df = pd.concat(concat_df, axis=1).reset_index(drop=True)
```

```
#jul_aug_df = pd.concat([jul_df, aug_df], axis=1).reset_index(drop=True)
```

```
print(jul_aug_df)
```

Result

	날짜	출금액	입금액	잔액	날짜	출금액	입금액	잔액
0	2023-07-01	80,000	0	100,000	2023-08-02	0	80,000	190,000
1	2023-07-01	0	150,000	250,000	2023-08-02	0	150,000	340,000
2	2023-07-05	75,500	0	174,500	2023-08-05	15,000	0	325,000
3	2023-07-11	3,000	0	171,500	2023-08-11	7,000	0	318,000
4	2023-07-11	0	15,000	186,500	2023-08-13	0	22,000	340,000

✓ merge()

- 두 DataFrame을 각 데이터에 존재하는 고유값(key)을 기준으로 병합할때 사용

```
import pandas as pd

age_sex_df = pd.read_csv('./나이_성별.csv')
add_df = pd.read_csv('./주소.csv')

merge_data = pd.merge(age_sex_df, add_df, on='이름')
#merge_data = pd.merge(age_sex_df, add_df, how='inner', on='이름')
print(merge_data)
```

Result

	이름	나이	성별	주소
0	김철수	32	남	안양시
1	최유리	30	여	서울시
2	신짱구	29	남	인천시
3	이훈이	29	남	서울시
4	정맹구	30	남	서울시

age_sex_df			
	이름	나이	성별
0	김철수	32	남
1	최유리	30	여
2	신짱구	29	남
3	이훈이	29	남
4	정맹구	30	남
5	신형만	63	남
6	봉미선	60	여

add_df	
	주소
0	안양시
1	서울시
2	인천시
3	서울시
4	서울시

✓ merge()

- 두 DataFrame을 각 데이터에 존재하는 고유값(key)을 기준으로 병합할때 사용

```
import pandas as pd

age_sex_df = pd.read_csv('./나이_성별.csv')
add_df = pd.read_csv('./주소.csv')

merge_data = pd.merge(age_sex_df, add_df, how='outer', on='이름')
print(merge_data)
```

Result

	이름	나이	성별	주소
0	김철수	32	남	안양시
1	최유리	30	여	서울시
2	신짱구	29	남	인천시
3	이훈이	29	남	서울시
4	정맹구	30	남	서울시
5	신형만	63	남	NaN
6	봉미선	60	여	NaN

age_sex_df

	이름	나이	성별
0	김철수	32	남
1	최유리	30	여
2	신짱구	29	남
3	이훈이	29	남
4	정맹구	30	남
5	신형만	63	남
6	봉미선	60	여

add_df

	이름	주소
0	김철수	안양시
1	최유리	서울시
2	신짱구	인천시
3	이훈이	서울시
4	정맹구	서울시

06 데이터 병합

✓ merge()

- how='left'는 첫번째 데이터프레임을 기준으로,
how='right'는 두번째 데이터프레임을 기준으로 병합함

age_sex_df				add_df	
	이름	나이	성별	이름	주소
0	김철수	32	남	0	김철수 안양시
1	최유리	30	여	1	최유리 서울시
2	신짱구	29	남	2	신짱구 인천시
3	이훈이	29	남	3	이훈이 서울시
4	정맹구	30	남	4	정맹구 서울시
5	신형만	63	남		
6	봉미선	60	여		

/*elice*/

```
merge_data = pd.merge(age_sex_df, add_df, how='left', on='이름')  
print(merge_data)
```

Result				
	이름	나이	성별	주소
0	김철수	32	남	안양시
1	최유리	30	여	서울시
2	신짱구	29	남	인천시
3	이훈이	29	남	서울시
4	정맹구	30	남	서울시
5	신형만	63	남	NaN
6	봉미선	60	여	NaN

```
merge_data = pd.merge(age_sex_df, add_df, how='right', on='이름')  
print(merge_data)
```

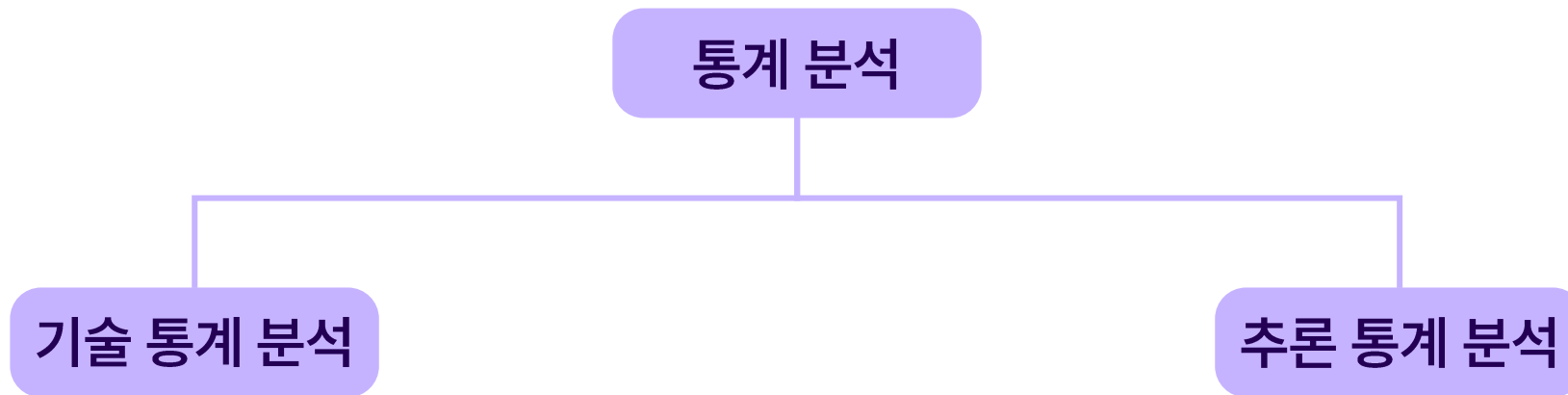
Result				
	이름	나이	성별	주소
0	김철수	32	남	안양시
1	최유리	30	여	서울시
2	신짱구	29	남	인천시
3	이훈이	29	남	서울시
4	정맹구	30	남	서울시

07

기초 통계

✓ 통계 분석

- 데이터에서 유용한 정보를 추출하기 위해 사용되는 방법
- 크게 기술 통계 분석과 추론 통계 분석으로 나눌 수 있음



✓ 기술 통계 분석

- 데이터를 간결하고 요약된 정보로 변환하여 데이터 세트에 대한 이해를 돕기 위해 사용됨
 - 요약된 정보는 수치적으로 나타낼 수 있으며 필요에 따라 시각화할 수 있음
- 가장 대표적인 기술 통계 방법으로는 평균값 계산, 분산 계산 등이 있음

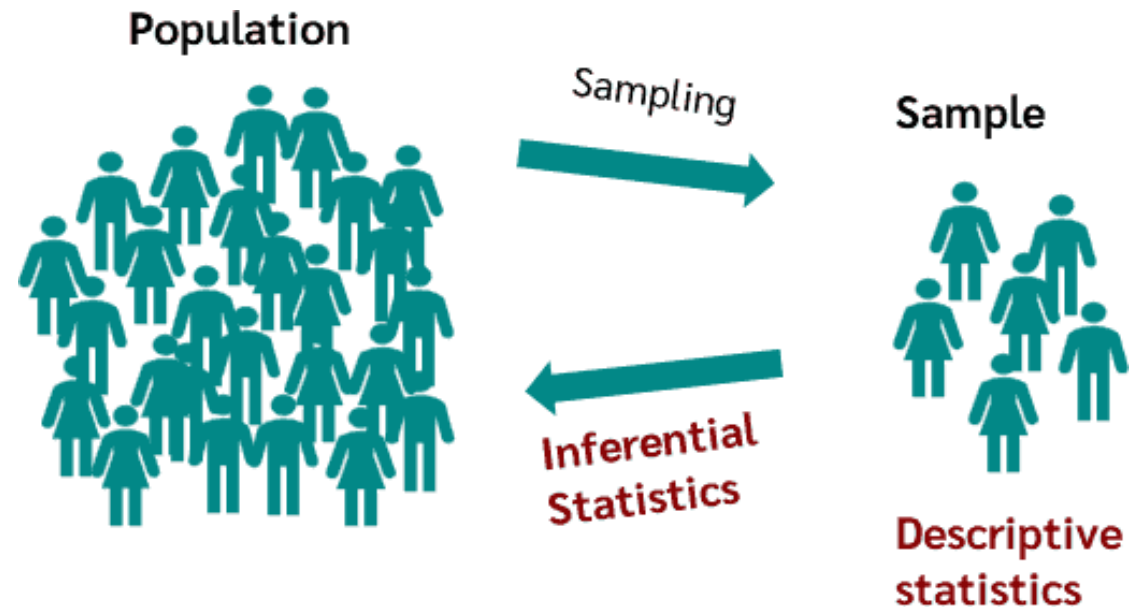
	수학성적	영어성적
학생 1	92	96
학생 2	85	78
...	...	...
학생 N	47	55



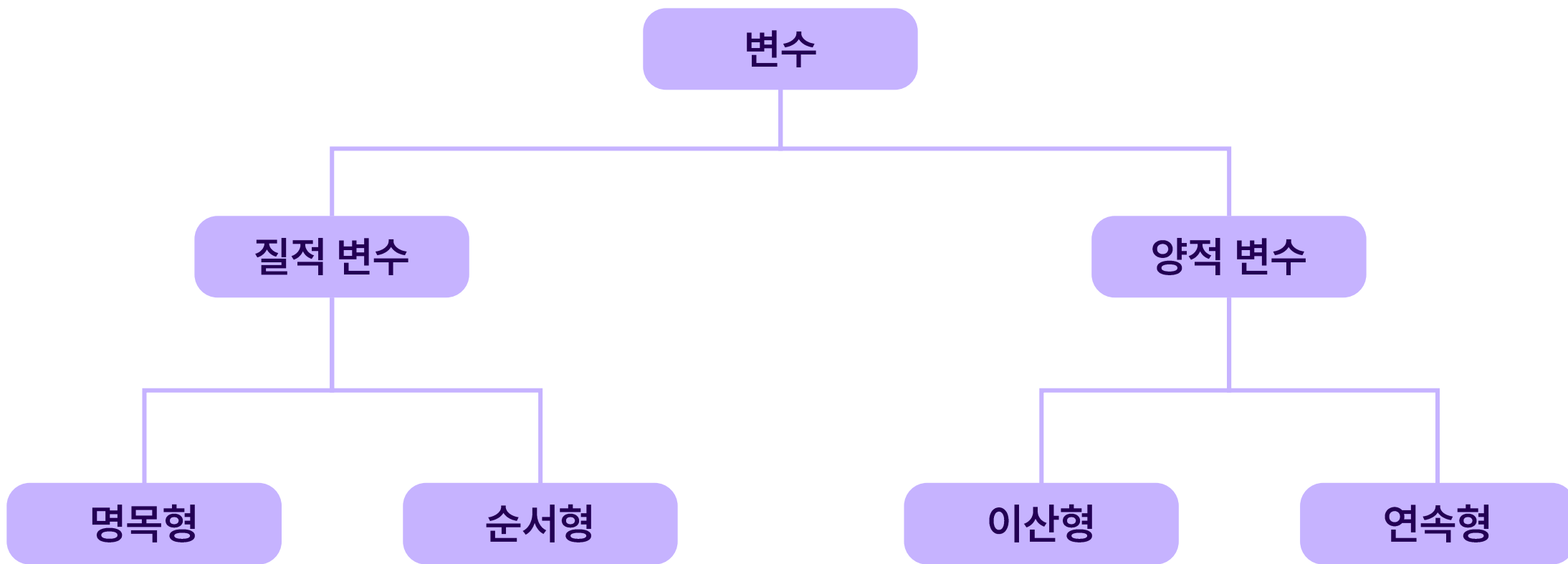
수학평균	영어평균
76	78

④ 추론 통계 분석

- 샘플 데이터(일부 데이터)로부터 전체 데이터를 추론하기 위한 통계
- 이러한 추론 통계의 가장 큰 전제는 선택된 표본이 모집단을 대표해야 한다는 것임



✓ 변수의 종류



✓ 평균값

- DataFrame에서 mean()를 이용해 양적변수에 해당하는 열에 대한 평균을 구할 수 있음

```
import pandas as pd

df = pd.read_csv('./train.csv')
print(df.mean())
```

Result	
PassengerId	446.000000
Survived	0.383838
Pclass	2.308642
Age	29.699118
SibSp	0.523008
Parch	0.381594
Fare	32.204208
dtype: float64	

✓ 중앙값

- DataFrame에서 median()를 이용해 양적변수에 해당하는 열에 대한 중앙값을 구할 수 있음

```
import pandas as pd

df = pd.read_csv('./train.csv')
print(df.median())
```

Result	
PassengerId	446.0000
Survived	0.0000
Pclass	3.0000
Age	28.0000
SibSp	0.0000
Parch	0.0000
Fare	14.4542

✓ 분산

- DataFrame에서 var()를 이용해 양적변수에 해당하는 열에 대한 분산을 구할 수 있음

```
import pandas as pd

df = pd.read_csv('./train.csv')
print(df.var())
```

Result	
PassengerId	66231.000000
Survived	0.236772
Pclass	0.699015
Age	211.019125
SibSp	1.216043
Parch	0.649728
Fare	2469.436846

✓ 표준편차

- DataFrame에서 std()를 이용해 양적변수에 해당하는 열에 대한 표준편차를 구할 수 있음

```
import pandas as pd

df = pd.read_csv('./train.csv')
print(df.std())
```

Result	
PassengerId	257.353842
Survived	0.486592
Pclass	0.836071
Age	14.526497
SibSp	1.102743
Parch	0.806057
Fare	49.693429